

Predicting Polarity in Political News Through Machine Learning Models:

Logistic Regression and BERT

Oscar J. Alvarado

School of Information, University of Texas at Austin

LIN 371: Machine Learning Text Analysis

Professor Elisa Ferracane

May 1, 2026

Introduction

For my final project, I decided to analyze sentences from political figures and determine their polarity. News is now an important part of our lives because information can travel so quickly today. And because of that, news sources can spread any kind of information, including about political figures. Because of this, I've decided to analyze sentences related to political figures and their polarity to create models that predict the polarity of sentences aligned with a political figure. This will help determine the values and opinions of news sources. In addition, this can help hold those accountable who portray a figure in a false manner. To do this, I will use three models: a baseline, logistic regression, and Bidirectional Encoder Representations from Transformers (BERT).

Dataset

The dataset I used to create these models came from HuggingFace and is called "NewsMTSC dataset". It contains the sentence normalized, targets, mention, polarity, and a few other features as well. This source is under a public domain license, meaning it may be used for public purposes and as a source for educational purposes. The training data set contains 7758 lines. The first test data is 721, and the second training data is 1067. I will focus on the sentence normalized and polarity to create the model.

Procedure

To create the models, I used a Jupyter notebook, which allows me to use Python and Markdown. To begin, I imported the dataset from my computer and determined the lengths of the training and test data. I followed this by printing the first ten lines of the data to show what it looks like. I then cleaned and extracted the data I will be analyzing. This included finding the correct feature from the dataset, lowercasing it, and then adding it to the correct list: sentences or polarities. In addition, since the polarities were 2, 4, and 6, I converted them to 0, 1, and 2 to better fit my model. I then used Numpy and Matplotlib to visualize the distribution of the polarities. I found that they were overall quite similar, with

negative at 37%, neutral at 36%, and positive at 27%. After this visualization, I defined my variables to ensure the same data is analyzed across all models. This included defining “docs_train, docs_temp, y_train, y_temp” as well as “docs_dev, docs_test, y_dev, y_test”. These two were required because I wanted to create three distinct sets of data: training, validation, and test. Training consisted of 70% of the data, 15% for deviation, and the last 15% for testing.

For my baseline, I used the majority-class baseline, which essentially predicts the most common polarity for all test samples. To do this, I used a package from Sklearn called the DummyClassifier. The strategy was called “most_frequent”. The next model was logistic regression, which also comes from Sklearn. Logistic regression is a linear model that doesn’t account for context and treats each word individually. It creates coefficients for each word and adds up the coefficients of each word in a sentence to determine the polarity. To follow this, I used Pandas and Numpy to identify the keywords most strongly associated with the polarity. The results are printed below.

Negative keywords: ['accused', 'guilty', 'trump', 'allegations', 'criticism', 'claims', 'arrested', 'resignation', 'charges', 'flynn', 'pelosi', 'alleged', 'saying', 'failing', 'anti', 'comey', 'attacks', 'sexual', 'lost', 'behavior']

Neutral keywords: ['said', 'told', 'asked', 'added', 'npr', 'wrote', 'mueller', 'talks', 'says', 'chairman', 'snowden', 'thought', 'want', '2018', 'monday', 'official', 'summit', 'continue', 'left', 'killed']

Positive keywords: ['support', 'work', 'right', 'working', 'great', 'good', 'proud', 'service', 'america', 'better', 'victory', 'help', 'loved', 'praised', 'life', 'strong', 'children', 'confident', 'focused', 'family']

I thought this was a good addition that shows which words contribute most to specific polarities. In addition, it helps see bias within the model. For example, “Trump” is associated mostly with negative, meaning that if a sentence were to have that name, it would most likely determine the polarity to be negative even if the context is positive. But this also shows how this name is associated with just being negative most of the time. In the neutral keywords, we can see that “npr” and “2018” are here, meaning that sentences containing these words or similar would be categorized as neutral, as these words are most

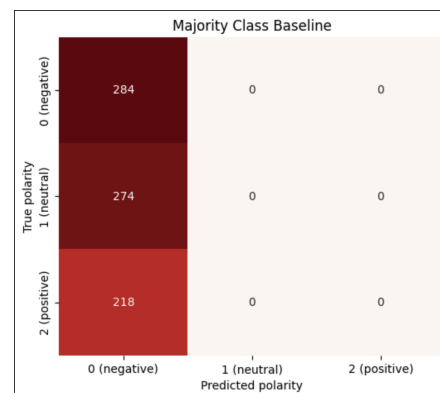
commonly used in factual sentences. The positive keywords incorporate common positive words like “great”, “proud”, and “victory”, but have also found that other words like “children” and “family” are mostly associated with positive. Finally, the last model used was BERT, which instead considers the context of a word by looking at the sentence as a whole. BERT uses its own tokenizer and uses Self Attention to see how words relate. This helps produce more accurate results than a logistic regression model, as it can evaluate when a positive word is used negatively, such as “not good”.

Results and Evaluation

For the baseline, which used the Sklearn package DummyClassifier, we predicted that each response would be the polarity most common in the dataset. Therefore, we had an accuracy score of .366, which is very close to the percentage of the negative polarity, .368, the most common polarity in the dataset. This was found by using the “most_frequent” strategy from the DummyClassifier. The difference of .002 is so small that this model remains consistent, as the data handling could have caused it. The recall score for negative polarity is 1.0 because when it predicted negative, which it did for all of them, it was correct. But for positive and neutral, the recall was 0.0 because it got it wrong all the time. The precision of .37 in the negative is because it was only right 37% of the time. The confusion matrix shows that the response was “negative” for all of them, visually indicating that none of the predictions were either positive or neutral, since the majority baseline only predicts the most common baseline, which was negative. And the macro f1-score of .18 shows how low the performance for all the polarities was.

Majority Class Baseline Accuracy: 0.3660

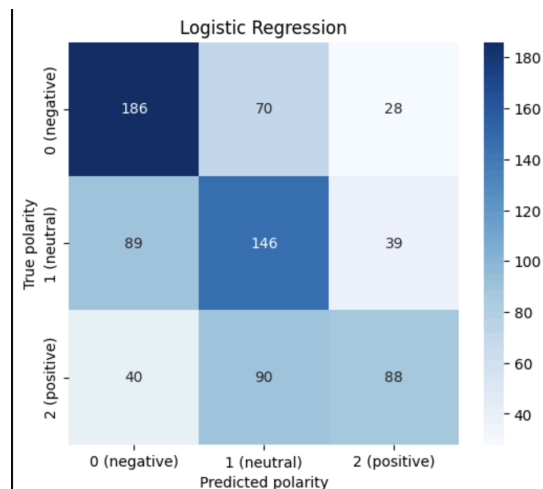
Baseline Classification Report:				
	precision	recall	f1-score	support
0	0.37	1.00	0.54	284
1	0.00	0.00	0.00	274
2	0.00	0.00	0.00	218
accuracy			0.37	776
macro avg	0.12	0.33	0.18	776
weighted avg	0.13	0.37	0.20	776



After this baseline, I used logistic regression to analyze the sentences. Positively, I definitely got higher scores than the baseline. The overall accuracy score was .54, a good jump from the baseline. This score is not as high as it could be because the logistic regression model does not account for context, so some predictions remain inaccurate. The neutral polarity has the least precision (.48), which means it's the one causing most of the inaccurate results. Positive had a precision of .57,, and negative had a precision of .59. The smallest recall score was for positive, at .40. This demonstrates that the model correctly identified 40% of the actual positives. The recall score for negative was .65, and for neutral it was .53. Positive also had the lowest f1-score, meaning that it failed to identify more than half of all the positive sentences. The scores for negative (.62) and neutral (.50) brought our macro F1-score up to .53. We should acknowledge that the positives remain the toughest to identify. The confusion matrix shows that other sentiments are also being used for predictions, unlike in our baseline. It shows visually how negative and neutral are being identified more accurately, not as much as desired, but more than the baseline, so it shows improvement. I included the “cbar” for this confusion matrix, as it can serve as a reference to show the majority.

```
lr_model accuracy is : 0.5412371134020618
classification report:
```

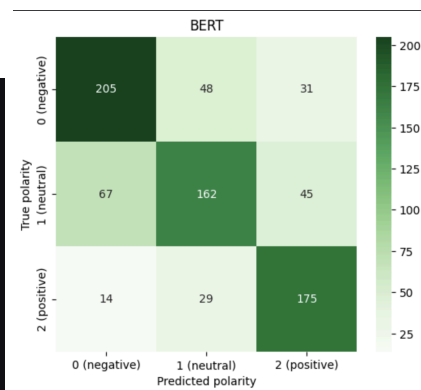
	precision	recall	f1-score	support
0	0.59	0.65	0.62	284
1	0.48	0.53	0.50	274
2	0.57	0.40	0.47	218
accuracy			0.54	776
macro avg	0.55	0.53	0.53	776
weighted avg	0.54	0.54	0.54	776



And finally, the BERT model showed much more improvement with an accuracy score of .70. The precision scores for all of these polarities were quite similar, with negative having a score of .72, neutral with .68, and positive with .70. This shows that overall, most of the predictions are on the right

track, but there is still a little room for improvement. Positive had the highest recall score of .80 meaning that 80% of the positives were accurately predicted. The lowest recall score was for neutral (.59). From the confusion matrix, we can see that the model had a hard time distinguishing neutral sentences and would most likely predict negative. The overall macro f1-score was .70, indicating the model is fairly well balanced and correctly classifies polarity most of the time. But, as stated before, the f1-score for neutral was the lowest at .63, showing again how neutral was the hardest to correctly identify. The confusion matrix visually shows how accurately the polarities are being identified.

Classification Report:				
	precision	recall	f1-score	support
0	0.72	0.72	0.72	284
1	0.68	0.59	0.63	274
2	0.70	0.80	0.75	218
accuracy			0.70	776
macro avg	0.70	0.71	0.70	776
weighted avg	0.70	0.70	0.70	776



Conclusion

These different models show how difficult it is to accurately determine the polarity of a sentence. Through this project, I have successfully implemented different models that have only increased accuracy scores. The baseline had the lowest accuracy score as it automatically predicted each sentence to be negative as that was the most common polarity throughout the dataset. Logistic regression improved but lacked context in order to be more accurate. And the BERT improved even more as it was able to use context to identify polarity. By creating these models,

References

GeeksforGeeks. (n.d.). Understanding feature importance in Logistic Regression models.

<https://www.geeksforgeeks.org/machine-learning/understanding-feature-importance-in-logistic-regression-models/>

Hamborg, F., & Donnay, K. (2021, April). NewsMTSC: (Multi-)target-dependent sentiment classification in news articles. In Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics (EACL 2021). Virtual Event.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

<https://scikit-learn.org/stable/modules/generated/sklearn.dummy.DummyClassifier.html>

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

W3Schools. (n.d.). Python Matplotlib pie charts.

https://www.w3schools.org/python/matplotlib_pie_charts.asp